The Shell

Definition

The **shell** is a command interpreter used in UNIX. It is **not part of the operating system itself**, but it uses many operating-system features and acts as an important interface between the user and the operating system.

1. Shell and Operating System

Important Idea

The operating system is the code that carries out **system calls**. Programs such as:

- editors,
- compilers,
- assemblers,
- linkers,
- utility programs,
- command interpreters

are useful, but they are **not part of the operating system**.

Why the Shell Matters

Although the shell is not part of the operating system, it makes heavy use of operating-system features. That is why it is a very good example of how system calls are used in practice.

2. What the Shell Does

Main Function

The shell is the main interface between:

- the **user**, and
- the **operating system**

unless the user is working through a graphical user interface (GUI).

Common UNIX Shells

Different UNIX shells exist, such as:

- `sh`
- `csh`
- `ksh`
- `bash`

All of them support the basic shell functionality.

3. How the Shell Starts

Login

When a user logs in, a shell is started automatically. The shell uses the terminal as:

- **standard input**
- **standard output**

Prompt

The shell begins by showing a **prompt**, often a dollar sign (`$`). The prompt tells the user that the shell is ready to accept a command.

4. Running a Command

Example

If the user types:

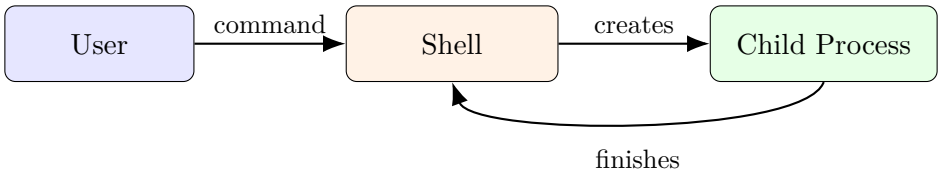
```
date
```

the shell creates a **child process** and runs the `date` program in that child.

What Happens Next

- The shell creates a child process.
- The child runs the requested program.
- The shell waits for the child to finish.
- After termination, the shell prints the prompt again.

4.1 Simple Command Flow Diagram



5. Output Redirection

Redirection

The user can redirect standard output to a file instead of sending it to the terminal.

Example

```
date >file
```

This sends the output of `date` to `file`.

6. Input and Output Redirection

Input + Output

Standard input can be taken from a file, and standard output can also be sent to another file.

Example

```
sort <file1 >file2
```

Here:

- `sort` takes input from `file1`
- the sorted result goes to `file2`

7. Pipes

Pipe

A **pipe** allows the output of one program to become the input of another program.

Example

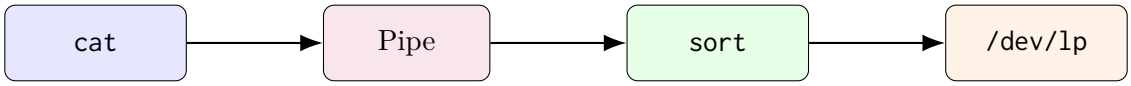
```
cat file1 file2 file3 | sort >/dev/lp
```

Explanation

In this command:

- `cat` combines the contents of three files,
- the output is passed through a pipe to `sort`,
- `sort` arranges the lines alphabetically,
- the final output is sent to `/dev/lp`, usually the printer.

7.1 Pipe Diagram



8. Background Jobs

Ampersand

If a command ends with an ampersand (`&`), the shell does **not wait** for that command to complete. Instead, it returns the prompt immediately.

Example

```
cat file1 file2 file3 | sort >/dev/lp &
```

This command runs as a **background job**.

Meaning

A background job allows the user to continue doing other work while the command is still running.

9. Shell vs GUI

Graphical User Interface

Most personal computers today use a **GUI** (Graphical User Interface). A GUI is also just a program running on top of the operating system, similar to the shell.

Examples

In Linux, users may choose:

- Gnome,
- KDE,
- or no GUI at all.

Key Idea

Both the shell and the GUI are interfaces for users. Neither of them is the operating system itself; both run on top of the operating system.